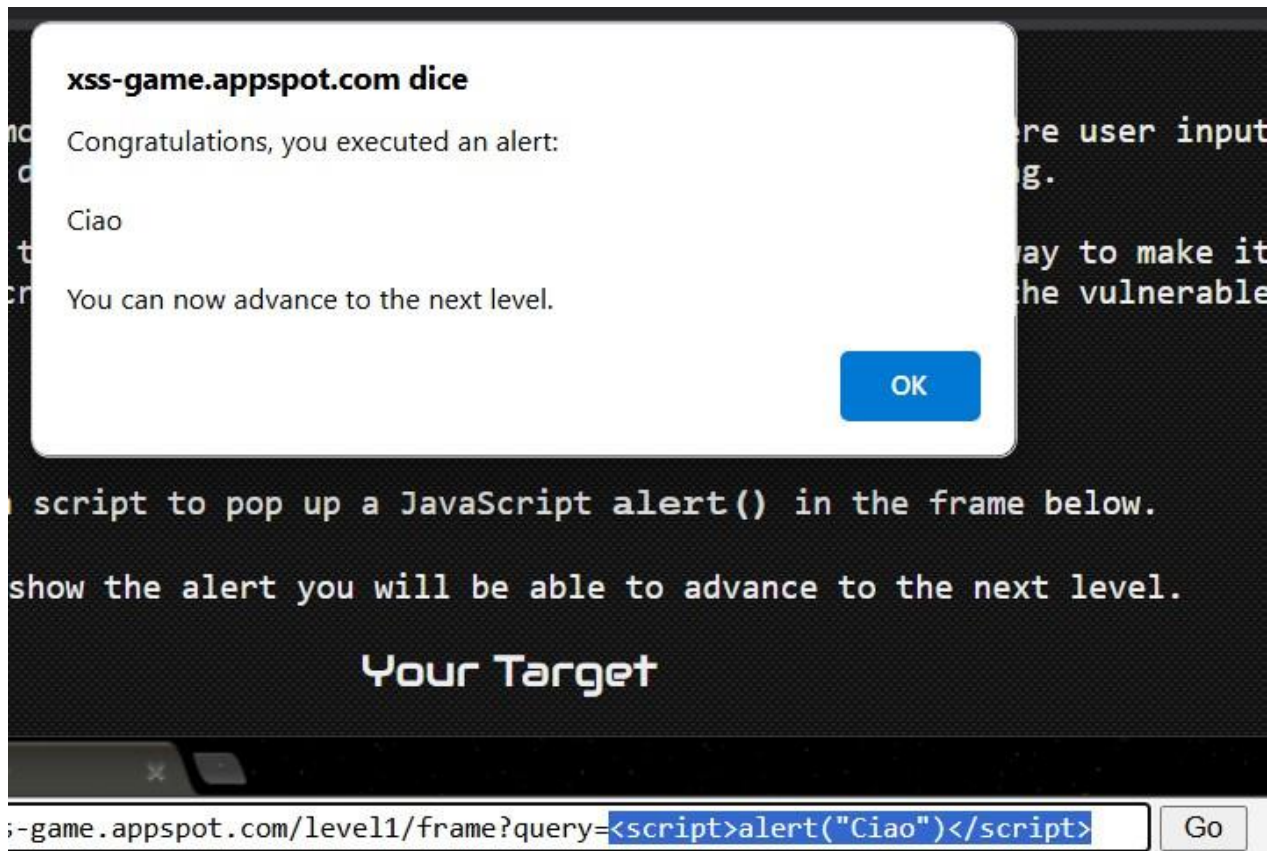


S7 L5 BONUS

Livello 1

Per effettuare un XSS (di tipo reflected in questo caso) possiamo semplicemente inserire nel form il seguente script: `<script>alert('Ciao')</script>`

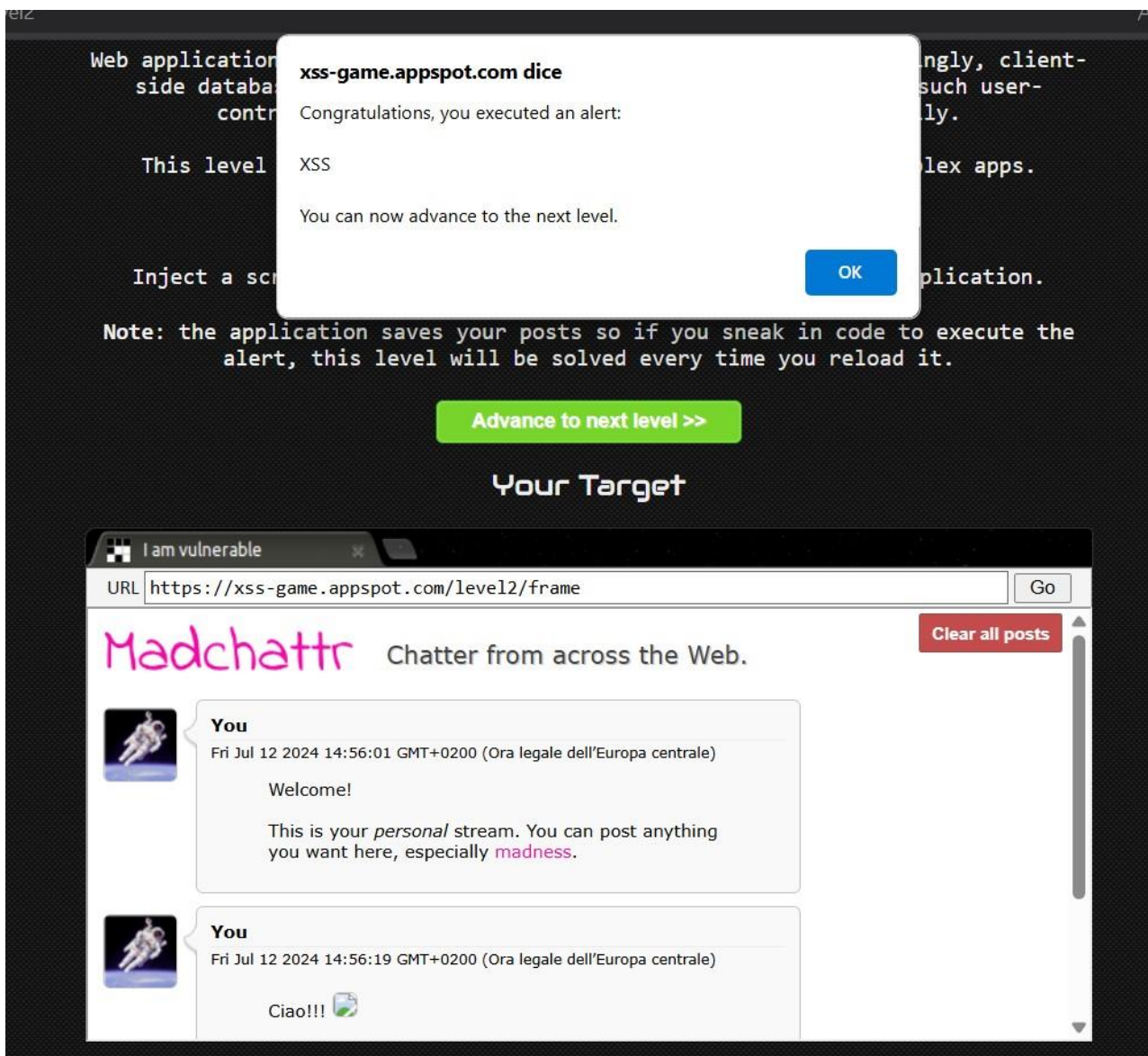


Sorry, no results were found for . [Try again.](#)

Non essendoci nessuna sanificazione dell'input possiamo semplicemente usare il tag script per eseguire codice malevolo

Livello 2

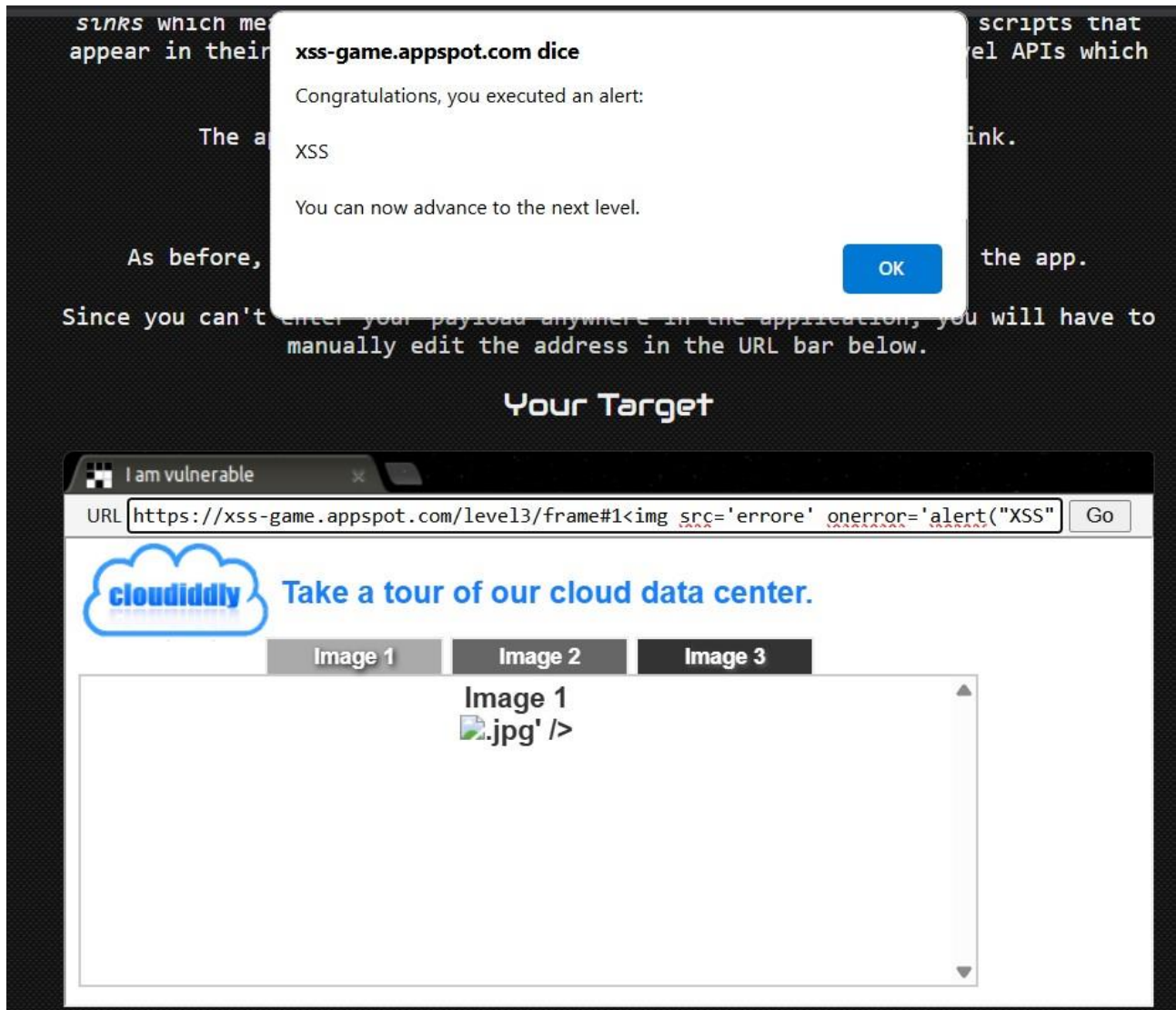
Qui parliamo di un XSS persistent. Il tag script non funziona, ma è possibile usare il tag image. Quello che possiamo fare è inserire un tag image con un URL invalido (passatogli tramite l'attributo **src**) e usare **onerror** che eseguirà un **alert** di javascript. Quello che succede è che essendo l'URL invalido restituirà un errore che viene gestito dall'attributo onerror che, in questo caso, avvierà il nostro **alert**. Lo script da inserire nel commento è `` Per coprire le tracce possiamo scrivere un commento davanti lo script. Come possiamo vedere lo script risulterà invisibile ad eccezione della broken image.



Livello 3

L'unico punto di iniezione nella pagina è l'URL. Per farlo possiamo provare ad inserire quanto fatto nel livello precedente ``

Il comando non sembra funzionare, ma se clicchiamo su una delle immagini notiamo che l'URL cambia andando ad aggiungere un # seguito dal numero dell'immagine. Se al nuovo URL aggiungiamo lo script precedente l'XSS viene iniettato.



Il motivo per cui funziona è da ricercare nel come viene gestito l'URL della pagina. Il codice che gestisce l'input dell'utente non valida e non sanifica adeguatamente il valore dell'input prima di utilizzarlo per costruire dinamicamente contenuti HTML. Questo fa sì che viene processato anche il tag image aggiunto che, come nel livello precedente, porta ad un errore e quindi all'esecuzione della funzione **alert** presente nell'attributo **onerror**.